

Computer Networks · Lab Report

Error Detection Codes

CRC & Checksum

*A Client–Server Simulation of Frame-Level Error Detection
using Cyclic Redundancy Check and Internet Checksum*

Name Anubhab Rakshit
Class BCSE–III
Roll No 002410501029

Assignment Lab Assignment 1
Domain Data Link Layer — Error Detection
Techniques CRC (8 / 10 / 16 / 32) & 16-bit Checksum
Architecture TCP Client–Server (localhost, port 6001)
Language C++ • Python (automation)

1. Purpose

The primary objective of this assignment is to implement and analyze common error-detecting codes used in the Data Link Layer, specifically **Cyclic Redundancy Check (CRC)** and **Checksum**. By developing a client-server architecture to simulate a network, we aim to inject errors into transmitted frames and evaluate the reliability and efficiency of both detection techniques in identifying corrupted payloads.

2. Project Structure

The project is architected into modular C++ components, cleanly separating functionality:

sender.cpp	Reads the data, segments it into frames, calculates CRC/Checksum, injects simulated errors, and transmits via sockets.
receiver.cpp	Listens for incoming socket connections, receives frames, parses them, and independently verifies CRC and Checksum to determine data integrity.
framing.hpp	Defines the TxFrame structure and handles payload segmentation and serialization.
crc.hpp / checksum.hpp	Core logical implementations of polynomial long division and 1's complement addition.
error_injection.hpp	Provides functions to artificially flip bits according to specific patterns (burst, single, two, odd).
network_utils.hpp	Helper functions for socket dial/accept and TCP stream management.
simulate.py	Python automation scripts used to run bulk test payloads and tabulate theoretical outcomes vs. practical detection.

3. Input & Output

Input

Raw data files (e.g. **sample.txt**, **batch_data.txt** of up to 20 KB). The user specifies the payload size, the CRC polynomial (**CRC-8**, **CRC-16**, **CRC-32**), and the desired error injection type (**single**, **two**, **odd**, **burst**).

Output

- A real-time log of frames arriving at the receiver, indicating **ACCEPT** or **REJECT** for both CRC and Checksum.

- A classification string for each frame: **BOTH_DETECTED**, **CRC_ONLY**, **CHECKSUM_ONLY**, or **NO_ERROR_DETECTED**.
- Batch performance metrics saved to a CSV and visualized as a summary chart (see Section 8).

4. Implementation

The implementation uses a standard **TCP Client–Server** model running on **localhost**. The **sender** acts as the client, establishing a connection to the **receiver** (server) on port **6001**. Data is broken down into maximum payload sizes (e.g. 64 bytes).

1. The **Checksum** is calculated by summing 16-bit words of the payload and taking the 1's complement.
2. The **CRC** is calculated by appending zero bits matching the degree of the polynomial, and performing XOR-based polynomial long division.
3. Errors are injected into the calculated bitstream before transmission over the socket.
4. The receiver performs the same calculations on the received payload and compares the expected bits with the received bits.

5. Code Snippets & Method Descriptions

Note — the snippets below are partial, high-level excerpts highlighting the core logic of each module.

crc.hpp • encodecrc()

```
// Polynomial long division using XOR
inline string encodecrc(const string &bits, const string &poly) {
    string g = polybits(poly);
    int r = g.size() - 1;
    vector<int> work(bits.size() + r, gen(g.size()));
    // ... (Convert string to int vectors)
    for (size_t i = 0; i < bits.size(); ++i)
        if (work[i])
            for (size_t j = 0; j < gen.size(); ++j)
                work[i + j] ^= gen[j]; // XOR operation
    // ... (Extract remainder)
    return bits + rem;
}
```

Description: This function calculates the CRC remainder. It takes the binary string and the polynomial, aligns the divisor, and performs modulo-2 division (XOR). It returns the original bits with the remainder appended.

checksum.hpp · checksum_enc()

```
// Calculates 16-bit Internet Checksum
inline vector<unsigned char> checksum_enc(const vector<unsigned char> &data) {
    uint32_t sum = 0;
    for (size_t i = 0; i < data.size(); i += 2) {
        uint16_t word = (data[i] << 8) + (i + 1 < data.size() ? data[i + 1] : 0);
        sum += word;
    }
    while (sum >> 16) {
        sum = (sum & 0xFFFF) + (sum >> 16); // Fold carries
    }
    uint16_t chk = ~sum; // 1's complement
    // ... (Return as bytes)
}
```

Description: Implements the classic Internet Checksum by treating data as 16-bit integers, summing them, adding the carry back into the lower 16 bits, and finally taking the one's complement.

error_injection.hpp · injecterr()

```
// Flips bits based on selected error profile
inline string injecterr(string bits, const string &kind, mt19937 &rng) {
    auto flip = [&](int i) { bits[i] = (bits[i] == '0') ? '1' : '0'; };
    if (kind == "burst") {
        // ... logic to flip a consecutive block of bits
    }
    // ...
    return bits;
}
```

Description: Takes the bitstream and randomly flips bits based on standard error models (single, burst, etc.) using a Mersenne Twister random number generator for reproducibility.

6. Test Cases

The system was tested against the following matrices:

- **Files:** Ten (10) randomized 20 KB files to ensure large sample sizes.
- **Polynomials:** CRC-8, CRC-10, CRC-16, CRC-32.

Error Injection	Description
single	1 bit flipped.
two	2 random bits flipped.
odd	1 or 3 random bits flipped.
burst	A consecutive block of up to 8 bits corrupted.

7. Assumptions & Reproducibility

Assumptions

The network medium (**localhost** TCP socket) is inherently reliable; therefore, all errors analyzed are strictly those injected artificially by **error_injection.hpp**.

Reproducibility

The sender accepts a **--seed** argument to initialize the **mt19937** RNG, meaning test runs can be perfectly replicated. Python scripts (**simulate.py**, **visualize.py**) have been provided to automatically compile, execute batch tests, and plot the charts in a single step.

8. Results & Findings

After executing the automated tests on 200 KB of randomized data, the following aggregated percentages were tabulated:

Batch Aggregated Error Detection Rate (%)

	single	two	odd	burst
CRC-8	100.00%	99.06%	100.00%	100.00%
CRC-10	100.00%	100.00%	100.00%	100.00%
CRC-16	100.00%	100.00%	100.00%	100.00%
CRC-32	100.00%	100.00%	100.00%	100.00%
Checksum	100.00%	97.96%	99.47%	100.00%

Theoretical vs. Practical Probabilities

Error Type	Observation
Single Bit	Theoretically, both Checksum and CRC should catch 100% of single-bit errors. The simulation matched this exactly.
Two Bit	Checksum can theoretically fail if bits in the exact same column position of two different 16-bit words are flipped in opposite directions. Detection rate dropped to ~98% in this scenario. CRC-8 showed marginal leakage; CRC-16 and CRC-32 caught 100%.
Burst	CRC detects all burst errors of length $\leq$ the degree of the polynomial. Since the burst limit was set to 8 bits, CRC-8 through CRC-32 achieved a 100% detection rate.

Analysis Methodology Findings

- CRC-16 and CRC-32 are significantly more robust against random noise and double-bit flips compared to Checksum.
- CRC computation (especially polynomial long division in software) is computationally heavier than the simple addition used by Checksum — making Checksum a favorable choice for higher-layer protocols (like TCP/UDP) where speed is prioritized over absolute reliability.

9. Implications for ARQ (Future Work)

This lab sets the fundamental error-detection layer needed for Reliable Data Transfer protocols. Because we can now confidently identify corrupted frames, the next step is to implement a mechanism to recover from them. As a foundation for the next assignment, we can implement **Go-Back-N (GBN) ARQ** or **Selective Repeat ARQ**:

- By adding sequence numbers and a timer to the **sender**.
- Having the **receiver** send NAKs (or silence) for frames that fail the CRC/Checksum check.
- The **sender** will then retransmit a window of frames (GBN) or just the specifically corrupted frame (Selective Repeat).

10. Challenges & Improvements

Challenges

Managing bit-level manipulation (converting bytes to strings of binary and back) in C++ was tedious and a source of initial bugs. Synchronizing the automated Python tester with the socket lifecycle of the C++ programs required careful timing constraints.

Improvements

The software CRC could be heavily optimized by using lookup tables rather than computing the XOR division bit-by-bit.

*Note: AI assistance (Google Gemini DeepMind) was utilized during this assignment to refactor the C++ architecture into modular files, develop the Python **simulate.py** batch-testing harness, generate data visualizations using pandas/matplotlib, and structure this markdown report.*